CS 1337 Computer Science I
Spring 2026 Syllabus UTD

INSTRUCTOR INFORMATION

Name	Jason Smith
Office	ECSS 3.232
Office Phone	972-883-4835

Office Hours:

Tuesday 2:00 – 4:00 PM
Thursday 10:00 AM - noon

Office hours are times that I am specifically available to meet with students. All office hours will be held on MS Teams. Please use the link provided in eLearning and log into your Teams account. Guests will not be admitted. When joining the link for office hours you will be placed in a queue, and I will admit you when it is your turn

Questions:

All questions must be in person (class time and office hours) or posted to Piazza (https://piazza.com/utdallas/spring2026/cs1337). **I will not answer questions by email.** Everyone is encouraged and expected to participate in answering questions and contributing to discussions in Piazza.

COURSE INFORMATION

Course Number	CE/CS 1337.003
Credit Hours	3
Meeting Time	M/W 10:00 – 11:15 AM
Room	ECSW 1.355

DO YOU NEED ASSISTANCE?

Help Desk: For help with issues regarding your computer, UTD maintains a walk-in help desk. Visit their Web site for details: http://www.utdallas.edu/ir/helpdesk/

If you need help, please make the effort to reach out. We can't help you if we don't know that you need help.

Resources:

- C++ language tutorial http://www.cplusplus.com/files/tutorial.pdf
- C++ reference: http://www.cppreference.com
- C++ tutorial http://www.learncpp.com/

Problem Solving Procedure

1. Try to solve it yourself – use the internet to research the problem and try different solutions. If you can't solve it after a couple of hours move to the next level.

2. Consult with your classmates – Post your question in Piazza or the class Discord server. On Piazza, you will be able to post your questions (anonymously if you wish) about anything related to the class and get a response either from me or a classmate. Private posts (that

only I can see) should be reserved for grade related questions or questions involving large snippets of code. Since there are multiple people that can answer questions, you should get a quick response allowing you to complete the tasks you are working on. If your question can't be answered there, move to the next level.

3. Visit the CSMC – If your classmates can't answer the question, check with a mentor at the CSMC (https://csmc.utdallas.edu). If the CSMC mentors can't answer your question, move to the next level.

4. Visit me during office hours – If you are truly stumped, I will give you the information you need to move forward.

WHAT DO I NEED FOR CLASS?

- **Zybooks (Required):**
  - CS 1337 – Computer Science I
  - You will pay for Zybooks when you enter the first assignment *(C-Strings and String Library Preview)*
  - All projects and homework assignments will be submitted through Zybooks for grading and plagiarism checking

- **Textbook (Optional)**:
  - Starting Out with C++, From Control Structures through Objects (9th or 10th edition); Gaddis, Tony; Pearson Publishing
    ISBN 9780137450626
    This is the same book that you may have used in 1336. It contains a few things not found in Zybooks. If you do not have this book already, you should be able to find it for very cheap as a used book. You may even try your luck at finding a digital version of it in the recesses of the internet.

- **C++ Compiler and IDE**
  - You may use any IDE you prefer.
    - It is best to use an IDE that does not identify or autocorrect syntax mistakes
    - This will help you in learning the language properly and performing well on the exams and programming quizzes
  - I encourage everyone to use Online GDB (https://www.onlinegdb.com)
    - This will simulate the ZyBooks environment for testing.
    - It provides a clean debugger interface that will make it easier to fix bugs in your code.
      - Please review the resources in eLearning for help with using the debugger
      - **Use of a debugger is one of the most important skills you should learn in this class**
      - A guide to using a debugger is provided in the Documents and Links section of eLearning
    - It's free and little set-up required.

  - All projects you submit will be compiled with g++ 5.4.1 with C++11 standards. You may use any IDE that can utilize at least g++ 5.4.1. Using a newer version of C++ should not create any problems as long as the proper standard is being used.

- **Note Taking Supplies** – I expect you to actively take notes. Anything discussed in class or available in the textbook is fair game on a test. Have something available to collect all the

knowledge given to you (be that in physical or digital form).

- **A computer** – Obviously, you need a computer for this class. Any computer bought off the shelf is sufficient for this class. However, if you do not have a computer, the CS computer lab is open 24/7 and contains all the software you will need for this class.

  I do not permit use of computers in class (except on in-class assignment days)**.** Tablets that use a digital pen (no keyboards or mice) are allowed for students to take notes. Exceptions will be made for those students that have accommodations from the Office of Accessibility.

  This is not a popular rule among students, but it is done so to maintain fairness in the academic environment for all students. There is no need to copy my code examples in class as I will make them available to you. I would rather you focus on how we are solving the problem and how we are utilizing the C++ tools to help us.

REQUIRED COURSE INFORMATION SECTION

Course Prerequisite: Prerequisite: CS 1436 with a grade of C or better or equivalent (placement test).

Description of Course Content: Review of control structures and data types with emphasis on structured data types. Applies the object-oriented programming paradigm, focusing on the definition and use of classes along with the fundamentals of object-oriented design. Includes basic analysis of algorithms, searching and sorting techniques, and an introduction to software engineering.

Student Learning Outcomes: After successful completion of this course, the student should have an:

- Ability to use single and multi-dimension arrays.
- Ability to implement linear and binary searches.
- Ability to implement simple sorting algorithms.
- Ability to implement structured data types.
- Ability to define and implement a class.
- Ability to use fundamentals of object-oriented design.

Attendance Policy: Grade deductions will be applied based on the number of classes missed for the semester as noted below:

- 1-5 absences – No deduction
- 6-10 absences – 1 letter grade drop (10 point deduction)
- 11 or more absences – automatic F in the class
- 0 absences – maybe extra credit?

TEACHING PHILOSOPHY

CS 1337 is skills-based class. Like any skill, learning requires practice and perseverance. You don't learn to play guitar by strumming a few chords you saw in a YouTube video. It takes time, energy, and dedication to the craft. It also requires a change in mindset compared to high school. Many see CS as fun and exciting because of a high school class that just taught the basics and made simple games. But CS is more about solving problems and implementing those solutions in a foreign language. The projects you will create in this class are inspired by actual problems from the professional world. You may feel that CS is no longer "fun" once exposed to these problems.

The goal of the class is not to learn C++, but to learn the concepts associated with object-oriented programming and use those concepts to solve problems. Anyone can learn a programming language on their own, but the language is simply a collection of tools. What you do with the tools is where the learning takes place. Just because you know how to use a hammer doesn't mean you can build a house.

I teach the class using a reverse classroom approach. This means that most of the learning will happen outside of the classroom. The classroom is where we will come together to help fill in the gaps and answer questions based on your experience outside of class. You will learn the concepts by completing activities and writing code. It is expected that you will share information (not solutions) with your peers which will solidify your understanding of the material.

My goal is to prepare you for what lies ahead so you will not only be ready for your remaining classes, but also ready to transition into the professional world. It's never too early to think about the things you need for the next phase of your life after graduation. In addition to learning foundational computer science concepts, students will improve their critical thinking, communication, debugging, and time management skills. All of these are necessary to be successful not only academically, but also professionally as well as in everyday life.

WHAT I EXPECT OF EACH STUDENT

- **Ask for help at any time.** If you do not understand something or are having trouble implementing a concept, follow the problem solving procedure defined on page 1. The sooner you ask that question, the sooner you will get an answer. That answer will allow you to move forward. I want you to succeed; don't be afraid to ask questions.

- **Take responsibility for your education.** I am not the type of teacher that lectures with slides and expects students to memorize them. I teach by creating program examples in class and asking you to participate in completing the code as well as ask questions about things you don't understand. I will give challenging assignments to push you toward learning concepts in-depth as well as developing critical thinking and core programming skills.

  Part of being a professional is learning how to teach yourself. I am going to guide you through the topics of the semester, but a significant portion of what you take with you to the next class will be things that you have learned on your own.

- **Practice time management skills.** Good time management is necessary for this class. All assignments (homework and projects) are designed to be worked on over a period of days or weeks. **I expect everyone to devote an average of 80 minutes a day (10 hours a week) working on things outside of class.** Remember that university and federal guidelines say that **for every credit hour of a course, you should spend a minimum of 3 hours for that class outside the classroom**. Doing this will help you to divide tasks up into chunks and work a little at a time on an assignment rather than waiting until a day or two before it is due. You will have a very difficult time succeeding in this class if you schedule to start and/or finish every assignment at the last minute.

- **Attend every class.** You are paying for an education. Don't waste your money by skipping class. I will give you everything you need to complete projects and do well on the tests. You have to be there to get the information.

- **Make mistakes!** This is how you learn. Don't be discouraged when something goes wrong. Programming takes lots of practice and mistakes will always happen. Study the mistakes you made so that you can learn from them for the future.

WHAT EACH STUDENT SHOULD EXPECT

- **A problem solving class.** This class is not a programming class. Computer science is all about problem solving. The content of this class is to teach you how to solve problems using a computer. In order to solve those problems, you will need to learn a foreign language (C++) and write solutions that the computer can interpret.

- **An open environment dedicated to learning.** I want students to feel free to voice their opinions. Oftentimes as we code in class, I will ask students what they would do in a certain situation. I want each student to feel as if he/she can speak freely and also be open for other students to discuss that idea, even if that means that some students will disagree.

- **Exams focused on syntax and application.** The programming language and how to use it are connected; each one is as important as the other. In the same way that knowing how to spell words does not make you a good writer, knowing the syntax of C++ does not mean that you know how to use those tools correctly. I will test both of these things in the exams. You will need to know the proper syntax for the programming concepts and how to use them properly. You will be expected to write code on the exam.

- **A simulated professional experience.** The projects in this class require you to exercise strategies found in "the real world". Your logic for a project may force you to learn new techniques that haven't yet been discussed in class. It is very likely that you will discuss logic with your classmates to better understand why your program isn't working. You will have to perform code maintenance and improve the efficiency of previously written code. These things offer a small taste of how life might be once you graduate and are given large sums of money by a company seeking your skills.

- **A deep understanding of C++ and object-oriented programming.** My goal is for you to know all the topics of CS 1337 so well that you do not need to look them up when trying to use them. I'm going to push you toward that goal. You should have peace of mind moving on in your degree plan because you will be fully prepared to tackle what the next course in the sequence will throw at you.

THE INFORMATION YOU REALLY CARE ABOUT

Grading Scale:

98-100 A+	88-89 B+	78-79 C+	68-69 D+	Below 60 F
92-97 A	82-87 B	72-77 C	62-67 D	
90-91 A-	80-81 B-	70-71 C-	60-61 D	

Grade Components:

Projects (3)	25%
Exams (2)	35%
Programming Quizzes	20%
Homework	10%
In-class Assignments	10%

General Grade Information: All grades will be available in eLearning. The Weighted Total column will give you the most accurate information concerning your grade. The weighted total is an approximation of your grade in the class based on the grades currently in eLearning.

I do not curve grades. Assignments are combined into categories so that a low grade for one item will not destroy your grade. Extra credit may be offered at the discretion of the professor

to everyone in the class. Any extra credit offered will require additional work and potentially be more complex than the scheduled assignments already. It will not be a free handout.

Grade Disputes: **All grade disputes must be reported within 1 week and resolved within 2 weeks of the grade in question being posted in eLearning.** Uncontested grades will become final after 1 week and cannot be disputed later. Announcements are made after each grade is posted so please check your grades promptly and reach out to the proper person.

I am responsible for grading your exams. If you have questions regarding your exam, please contact me through a private post on Piazza.

Everything else will be graded by a TA. Please address any grading concerns you have regarding these grades with the TA. **Please make a post only to instructors in Piazza if you have a grading question.**

You are responsible for verifying that all grades in Zybooks are recorded in eLearning correctly. Once you submit your assignment in ZyBooks, remember to press the Submit to Blackboard button. Then, check My Grades in eLearning to check that the correct grade was transferred.

Projects: Projects will be major programming assignments that supplement recently discussed topics and will be completed in two to three weeks. Projects are intended to take approximately 15-20** hours to complete overall; this includes the design, coding and testing process. Waiting until a couple of days before the due date to start the project is a bad idea. Not only does this introduce unnecessary stress into your life, it hardly ever ends well for the student. Most students score poorly on projects that are built in less than three days.

*(** this is average time for a student that has taken 1336 at UTD. Your experience with problem solving in computer science may impact the time needed to complete assignments)*

Projects will be divided into milestones as you would expect to see in the professional world. The milestones for each project are as follows:

- Design (4 days)
- Core implementation (7-10 days)
- Final implementation and testing (7-10 days)

Each milestone will be graded and the total of all three milestones will contribute toward the overall grade for the project. Each milestone will have a firm deadline and failure to meet the deadlines will have a negative impact on your grade.

All coding assignments (projects and review homework) are individual endeavors and students are not to work in groups. Students are permitted (and I openly encourage students) to discuss the logic of the assignment, but **DO NOT SHOW YOUR CODE TO OTHER STUDENTS.** When discussing logic, try to keep it general. If you give out every little piece of logic you have, there is a good chance the person you are helping will have very similar code as yours and may be flagged for being too similar. Be careful of posting your code online in public repositories. Another student could use your code without your knowledge and could involve you in a plagiarism referral.

Students should avoid using web sites like GitHub and Chegg for help on projects. Copying code from a web site is considered plagiarism and will be treated as such. If you find code on a web site, it is highly likely another student will find it as well which may cause both submissions to be flagged for similarity. Submitting project and assignment details to web sites for outside help (e.g. Chegg) is also considered academic dishonesty by UTD.

All projects will be submitted in ZyBooks and will be compared for originality. Any projects that are approximate or identical copies will be reported to the Office of Community Standards and Conduct, and I will accept their decision in regard to the grade if they believe that academic dishonesty has occurred.

Projects will be graded on a 100 point basis. Not only will your project be graded on proper execution, but also things like efficiency, implementation and documentation. Keep in mind that you always want to write code that is easy to understand and is also easy to maintain. Fewer lines do not necessarily mean a better program. Please use comments liberally.

You are responsible for testing your project thoroughly before submission. I will not give you the exact test cases that will be used for grading before the project is due. As a computer scientist, you must be able to identify all possible input and make sure that your code produces proper output and does not crash.

Late Projects:

All project milestones will be due at 11:59 PM on the day listed in the project documentation. The final project code will be accepted up to 24 hours late with the following penalties

<= 1 hour	-5 points
1 – 3 hours	-10 points
3 – 12 hours	-25 points
12 – 24 hours	-50 points

Preview Homework: Preview homework assignments are activities based on the readings in ZyBooks. Students will read through the sections and answer basic questions about the material to illustrate they understand the general concepts.

Review Homework: Review homework assignments are generally short coding assignments that can be done in 1-2 hours and measure how well you understand the material we have covered. These assignments are typically due 1 week from the date given.

Exams: Exams will cover chapters as listed below in the tentative course schedule. Exams will primarily consist of multiple choice/answer questions. An exam should not be missed except for the most extreme. If you miss an exam, you must have documentation for the absence. A make-up exam may be given to students with valid documentation. The allowance of a make-up exam is at the sole discretion of the instructor.

All exams and quizzes will be given online in eLearning. Students are expected to take the exams and quizzes at the Testing Center on the day scheduled during the window of opportunity provided by the Testing Center. The exams will be closed book and closed notes. Quizzes will be open book and closed notes. All solutions to coding quizzes will be submitted for similarity to maintain academic integrity.

Students must have a reservation at the Testing Center to take the exams and quizzes. Reservations must be made at least 48 hours in advance. For example, if you wish to take an exam at noon on Friday, the reservation must be submitted before noon on Wednesday. Please note that availability for the exam will fill up quickly. I strongly encourage you to make all of your reservations at the beginning of the semester. **I will not grant a make-up exam because a student forgot to make a reservation. It is your responsibility to confirm your reservation based on the email sent by the Testing Center. No email means no reservation.**

You are expected to be at the Testing Center at your scheduled time. You will not be admitted into the Testing Center if you are more than 15 minutes late. Make-up exams will not be given if you are late to the exam unless there is an emergency situation.

Assignment Due Date Exceptions: In general, assignments are not accepted late except for the final submission of a project (see late penalties above). However, I know that life has a way of bringing the unexpected at the most inopportune times. If you have an exceptional life situation (personally or academically) that is creating difficulty for you to meet the given due date of an assignment, I will work with you to give an extension as long as you contact me **at least 24 hours before the due date**.

ARE WE THERE YET?

All dates are subject to change at the discretion of the instructor.

Date	Topic	
1/21	Introduction to C	C++ Concepts in C (eLearning)
1/26	C-strings	Chapter 3
1/28	**Last day to add/swap classes**	
	Characters and the String Class	Chapter 3
2/2	**In-Class Assignment #1**	
2/4	Recursion	Chapter 4
2/5	**Last day to withdraw without "W"**	
2/9	Recursion	Chapter 4
2/7	**In-Class Assignment #2**	
2/11	Searching Arrays	Chapter 5
2/13	**Programming Quiz #1**	
2/16	Sorting Arrays	Chapter 5
2/18	Pointers	Chapter 6
2/23	Pointers in C	Chapter 7
2/25	Smart Pointers	Chapter 6
3/2	**In-class Assignment #3**	
3/4	Structures	Chapter 8
3/6	**Programming Quiz #2**	
3/9	Linked Lists	Chapter 8 Linked list resources (eLearning)
3/11	Linked Lists	
3/13	**Midterm Exam**	
3/16 – 3/22	**SPRING BREAK**	
3/23	Linked Lists	Chapter 8
3/25	Enumerated Data Types	Chapter 8
3/30	**In-class Assignment #4**	

4/1	Introduction to Classes	Chapter 9
4/3	**Programming Quiz #3**	
4/6	Introduction to Classes	
4/8	Introduction to Classes Copy Constructor	
4/9	**Last day to drop**	
4/13	Overloaded Operators	Chapter 9
4/15	**In-class Assignment #5**	
4/20	Overloaded Operators	Chapter 9
4/22	Overloaded Operators	
4/27	**In-class Assignment #6**	
4/29	Inheritance	Chapter 10
5/1	**Programming Quiz #4**	
5/4	Polymorphism	Chapter 10
5/6	**In-class Assignment #7**	
5/11	**Final Exam**	

Assignment Calendar

All assignments due by 11:59 on the due date listed unless otherwise noted
All dates are subject to change at the discretion of the instructor.
Please refer to eLearning for any updates.
All submissions are made using the provided links in eLearning

Assignment	Post Date	Due Date
C-Strings and String Library Preview	1/21	1/26
Recursion Preview	1/21	1/29
Review Homework 1	1/26	2/1
Review Homework 2	2/2	2/8
Searching and Sorting Preview	1/21	2/10
Pointers (C++ and C) Preview	1/21	2/17
Project 1 Pseudocode	2/23	2/27
Project 1 Core Implementation	2/23	3/5
Structures and Enumeration Preview	1/21	3/5
Midterm Exam	3/14	3/13
Project 1 Final Submission	2/23	3/15
Project 2 Pseudocode	3/23	3/27
Review Homework 3	3/23	3/29

Objects and Classes Preview – Part I	1/21	3/31
Project 2 Core Implementation	3/23	4/3
Project 2 Final Submission	3/23	4/10
Objects and Classes Preview – Part II	1/21	4/7
Objects and Classes Preview – Part III	1/21	4/16
Project 3 Pseudocode	4/13	4/17
Review Homework 4	4/13	4/19
Review Homework 5	4/15	4/26
Inheritance and Polymorphism Preview	1/21	4/28
Project 3 Final Submission	4/13	5/3
Review Homework 6	5/1	5/8
Final Exam	5/11	5/11

University Policies
For all other University policies, please visit http://go.utdallas.edu/syllabus-policies